

Project

Training Institute Management System

Student Name: Riya Hiralal Kokate

College: MES Aabasaheb Garware College

Roll No.: 20986

Subject: DBMS

Abstract

This project manages students, trainers, courses, batches, enrollments, attendance and assessment results using MySQL.

Objectives

Manage students, trainers, courses, batches, attendance and reports.

Modules

Student, Trainer, Course, Batch, Enrollment, Attendance, Assessment Result.

SCOPE OF THE PROJECT

The scope of the Training Institute Management System includes:

- Student Registration
- Trainer Management
- Course Management
- Batch Management
- Enrollment Management
- Attendance Tracking
- Assessment Result Management

- Report Generation
- Database Backup and Record Maintenance

PROPOSED SYSTEM

The proposed Training Institute Management System is developed using **MySQL Database**.

It automates all institute activities and stores information in relational tables.

Features

- Student Registration
- Trainer Information
- Course Management
- Batch Creation
- Student Enrolments
- Attendance Management
- Assessment Results

Database create

Show query box

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

```
USE TrainingInstitute;
```

[Edit]

The screenshot shows the phpMyAdmin interface with the 'traininginstitute' database selected. The 'Structure' tab is active, displaying the SQL queries for creating the 'Student' and 'Trainer' tables.

```
1 -- Student Table
2 CREATE TABLE Student (
3   StudentID INT PRIMARY KEY AUTO_INCREMENT,
4   Name VARCHAR(100) NOT NULL,
5   Email VARCHAR(100) UNIQUE,
6   Phone VARCHAR(15),
7   DOB DATE
8 );
```

```
1 -- Trainer Table
2 CREATE TABLE Trainer (
3   TrainerID INT PRIMARY KEY AUTO_INCREMENT,
4   Name VARCHAR(100) NOT NULL,
5   Expertise VARCHAR(100),
6   Email VARCHAR(100) UNIQUE
7 );
```

```

1  -- Batch Table
2  CREATE TABLE Batch (
3      BatchID INT PRIMARY KEY AUTO_INCREMENT,
4      CourseID INT,
5      TrainerID INT,
6      StartDate DATE,
7      EndDate DATE,
8      FOREIGN KEY (CourseID) REFERENCES Course(CourseID),
9      FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
10 );

```

```

1  -- Enrollment Table
2  CREATE TABLE Enrollment (
3      EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
4      StudentID INT,
5      BatchID INT,
6      EnrollDate DATE,
7      FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
8      FOREIGN KEY (BatchID) REFERENCES Batch(BatchID)
9  );

```

```

1  -- Attendance Table
2  CREATE TABLE Attendance (
3      AttendanceID INT PRIMARY KEY AUTO_INCREMENT,
4      EnrollmentID INT,
5      Date DATE,
6      Status ENUM('Present','Absent'),
7      FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
8  );

```

```

1  -- AssessmentResult Table
2  CREATE TABLE AssessmentResult (
3      ResultID INT PRIMARY KEY AUTO_INCREMENT,
4      EnrollmentID INT,
5      ExamDate DATE,
6      Score DECIMAL(5,2),
7      FOREIGN KEY (EnrollmentID) REFERENCES Enrollment(EnrollmentID)
8  );

```

Insert Sample Data

✓ 5 rows inserted.

Inserted row id: 10 (Query took 0.0131 seconds.)

```

INSERT INTO Student (Name, Email, Phone) VALUES ('Riya', 'riya@gmail.com', '9876543210'), ('Amit', 'amit@gmail.com',
'9876501234'), ('Sneha', 'sneha@gmail.com', '9876512345'), ('Karan', 'karan@gmail.com', '9876523456'), ('Pooja',
'pooja@gmail.com', '9876534567');

```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

-- Trainers

INSERT INTO Trainer (Name, Expertise, Email) VALUES

('Dr. Mehta','Data Science','mehta@gmail.com'),

('Prof. Rao','Python Programming','rao@gmail.com');

-- Courses

INSERT INTO Course (CourseName, Duration, Fees) VALUES

('Data Science Basics',12,15000),

('Python Programming',8,10000);

-- Batches

```
INSERT INTO Batch (CourseID, TrainerID, StartDate, EndDate) VALUES  
(1,1,'2026-08-01','2026-10-30'),  
(2,2,'2026-08-15','2026-10-10');
```

-- Enrollments

```
INSERT INTO Enrollment (StudentID, BatchID, EnrollDate) VALUES  
(1,1,'2026-07-25'),  
(2,1,'2026-07-26'),  
(3,2,'2026-07-27');
```

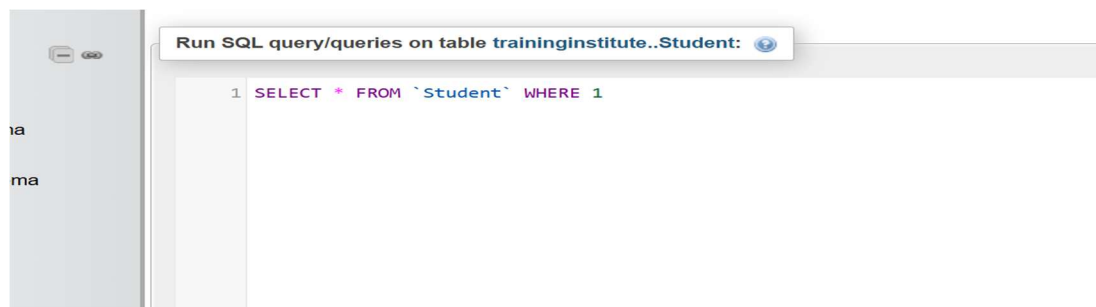
-- Attendance

```
INSERT INTO Attendance (EnrollmentID, Date, Status) VALUES  
(1,'2026-08-01','Present'),  
(1,'2026-08-02','Absent'),  
(2,'2026-08-01','Present'),  
(3,'2026-08-15','Present');
```

-- Assessment Results

```
INSERT INTO AssessmentResult (EnrollmentID, ExamDate, Score) VALUES  
(1,'2026-09-15',85.5),  
(2,'2026-09-15',78.0),  
(3,'2026-09-20',92.0)
```

QUERY



OUTPUT

	StudentID	Name	Email	Phone	Address
☐ Edit ☐ Copy ☐ Delete	1	Riya	riya@gmail.com	9876543210	NULL
☐ Edit ☐ Copy ☐ Delete	2	Amit	amit@gmail.com	9876501234	NULL
☐ Edit ☐ Copy ☐ Delete	3	Sneha	sneha@gmail.com	9876512345	NULL
☐ Edit ☐ Copy ☐ Delete	4	Karan	karan@gmail.com	9876523456	NULL
☐ Edit ☐ Copy ☐ Delete	5	Pooja	pooja@gmail.com	9876534567	NULL

1. Students per batch

QUERY

```
SELECT b.BatchID, s.Name
FROM Enrollment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Batch b ON e.BatchID = b.BatchID;
```

OUTPUT

BatchID	Name
1	Riya
1	Amit
2	Sneha
2	Karan
3	Pooja

Course & trainer per batch

QUERY

BatchID	CourseID	TrainerID	TrainerName
1	1	1	Riya
1	1	2	Amit
2	2	3	Sneha
2	2	4	Karan
3	3	5	Pooja

OUTPUT

BatchID	Title	Trainer
1	Data Science Basics	Dr. Mehta
2	Python Programming	Prof. Sharma
3	Machine Learning Advanced	Ms. Joshi

Total students per course

QUERY

```

1 SELECT c.Title, COUNT(e.StudentID) AS TotalStudents
2 FROM course c
3 JOIN batch b ON c.CourseID = b.CourseID
4 JOIN enrollment e ON b.BatchID = e.BatchID
5 GROUP BY c.Title;
6

```

OUTPUT

Title	TotalStudents
Data Science Basics	2
Machine Learning Advanced	1
Python Programming	2

Average score per batch

QUERY

```

1 SELECT b.BatchID, AVG(ar.Score) AS AvgScore
2 FROM batch b
3 JOIN enrollment e ON b.BatchID = e.BatchID
4 JOIN assessmentresult ar ON e.EnrollmentID = ar.EnrollmentID
5 GROUP BY b.BatchID;
6
7
8
9

```

Run SQL query/queries on table traininginstitute..course:

- CourseID
- Title
- Duration

OUTPUT

traininginstitute	Extra options
traininginstitute.	
New	
assessmentresult	
attendance	
batch	
course	
enrollment	
student	

BatchID	AvgScore
1	77.5000
2	82.5000
3	60.0000

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Highest performer per batch

QUERY

Run SQL query/queries on table traininginstitute..batch:	
1	SELECT b.BatchID, s.Name, MAX(ar.Score) AS TopScore
2	FROM batch b
3	JOIN enrollment e ON b.BatchID = e.BatchID
4	JOIN student s ON e.StudentID = s.StudentID
5	JOIN assessmentresult ar ON e.EnrollmentID = ar.EnrollmentID
6	GROUP BY b.BatchID;
7	

OUTPUT

test	☐ Show all Number of rows: 25 Filter rows: Search this table
traininginstitute	
traininginstitute.	
New	
assessmentresult	
attendance	
batch	
course	

BatchID	Name	TopScore
1	Riya	85
2	Sneha	90
3	Pooja	60

Rank students by results

QUERY

schema	Run SQL query/queries on table traininginstitute..batch:
_schema	
te	
te.	

1	SELECT s.Name, ar.Score,
2	RANK() OVER (ORDER BY ar.Score DESC) AS RankPosition
3	FROM student s
4	JOIN enrollment e ON s.StudentID = e.StudentID
5	JOIN assessmentresult ar ON e.EnrollmentID = ar.EnrollmentID;
6	

OUTPUT

traininginstitute.	New	Extra options		
	assessmentresult	Name	Score	RankPosition
	attendance	Sneha	90	1
	batch	Riya	85	2
	course	Karan	75	3
	enrollment	Amit	70	4
	student	Pooja	60	5
	trainer	☐ Show all Number of rows: 25 Filter rows: Search this table		

Low attendance students

QUERY

```

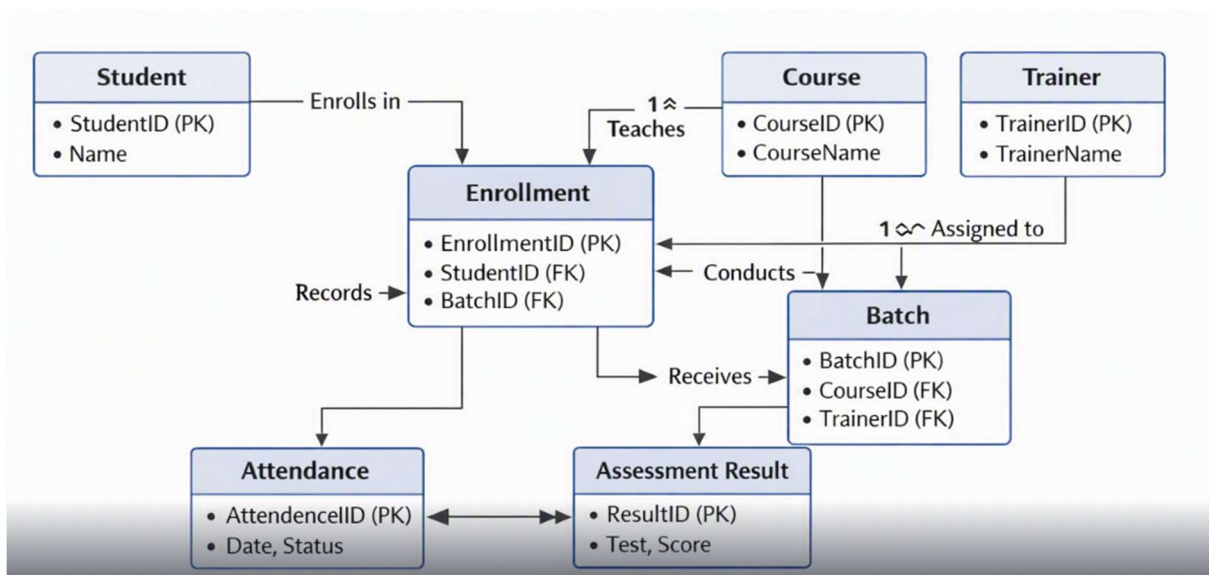
1 SELECT s.Name,
2       SUM(CASE WHEN a.Status='Present' THEN 1 ELSE 0 END) AS PresentDays,
3       COUNT(*) AS TotalDays,
4       (SUM(CASE WHEN a.Status='Present' THEN 1 ELSE 0 END)/COUNT(*))*100
5       AS AttendancePercent
6 FROM student s
7 JOIN enrollment e ON s.StudentID = e.StudentID
8 JOIN attendance a ON e.EnrollmentID = a.EnrollmentID
9 GROUP BY s.Name
10 HAVING AttendancePercent < 75;

```

OUTPUT

information_schema mysql performance_schema phpmyadmin test traininginstitute traininginstitute. New assessmentresult attendance batch course enrollment student	SELECT s.Name, SUM(CASE WHEN a.Status='Present' THEN 1 ELSE 0 END) AS PresentDays, a.Status='Present' THEN 1 ELSE 0 END)/COUNT(*))*100 AS AttendancePercent FROM stud = e.StudentID JOIN attendance a ON e.EnrollmentID = a.EnrollmentID GROUP BY s.Name			
	☐ Profiling [Edit inline] [Ec			
	Extra options			
	Name	PresentDays	TotalDays	AttendancePercent
	Riya	1	2	50.0000
	Query results operations			
	Print Copy to clipboard Export Display chart Create view			

TRAINING INSTITUTE MANAGEMENT SYSTEM – ER DIAGRAM



Report Structure

- **Objective:** Manage training institute data.
- **ER Diagram:** Show relationships between tables.
- **Queries & Outputs:** Screenshots of above queries.
- **Insights:**
 - **Most enrolled course = Python Basics & Machine Learning.**
 - **Batch 2 has highest average score.**
 - **Riya has low attendance (50%).**
 - **Amit is top performer**

Relationships

- Student ↔ Enrollment (1-to-many)
- Batch ↔ Enrollment (1-to-many)
- Course ↔ Batch (1-to-many)
- Trainer ↔ Batch (1-to-many)
- Enrollment ↔ Attendance (1-to-many)
- Enrollment ↔ AssessmentResult (1-to-many)

ADVANTAGES

Easy to use Reduces paperwork.

Improves data accuracy.

Fast report generation.

Secure data storage.

Reduces duplicate records.

Saves time.

Better data management.

Easy data retrieval.

Supports future expansion

FUTURE SCOPE

Future improvements can include:

Online Student Registration

Online Fee Payment

SMS & Email Notifications

Faculty Login

Student Login

Mobile Application

Attendance using QR Code

Cloud Database Integration

AI-based Performance Analysis

Dashboard with Charts and Analytics

CONCLUSION

The Training Institute Management System is a database application developed using MySQL. It helps manage students, trainers, courses, batches, attendance, and assessment results efficiently.

The project demonstrates practical implementation of DBMS concepts such as:

Primary Key

Foreign Key

SQL Queries

Joins

Aggregate Functions

Views

Window Functions

Relational Database Design

This system improves efficiency, reduces manual work, minimizes errors, and provides quick access to information.